# -: I2C Protocol :-

## 01. Master in Read Mode (only one byte) :-

```
/* Master in Read Mode*/

#include<avr/io.h>
void i2c_init (void){
TWSR = 0X00; // set prescaler bits to zero
TWBR = 0x47; // SCL frequency is 50K for XTAL = 8Mhz
TWCR = 0x04; // enable TWI module
}

void i2c_start(void){
 TWCR = ( 1 << TWINT ) | ( 1 << TWSTA ) | ( 1 << TWEN
);
 while ((TWCR & ( 1 << TWINT ) ) == 0) ;
}

void i2c_write(unsigned char data){
 TWDR = data ;
 TWCR = (1<<TWINT) | (1<<TWEN); // enable the
transmission
 while ((TWCR & ( 1 << TWINT ) ) == 0); // polling e.g
checking TWINT flag to check whether START condition
transmitted succesfully
}

unsigned char i2c_read(unsigned char last){
 if ( last == 0 ) // If want to read more than 1 byte
    TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
 else           // else read only one byte
 TWCR = (1<<TWINT) | (1<<TWEN);
 while ((TWCR & ( 1 << TWINT ) ) == 0);
 return TWDR ;
}

void i2c_stop(){
TWCR = ( 1 <<TWINT )|( 1 << TWEN) | ( 1<< TWSTO);
}

int main(void){
unsigned char i = 0 ;
DDRD = 0xFF; // PORTD is output
i2c_init(); // initialse TWI for master mode
i2c_start(); // transmit start condition
i2c_write(0b11010001); // transmitt Slave address +
R(1) (SLA+R means to master is reading )
i = i2c_read(1); // read only 1 byte
PORTD= i; // show byte in PORTD
i2c_stop(); // transmitt stop condition
while(1);
return 0;
}
```

## 02. Master in Write Mode (only one byte) :-

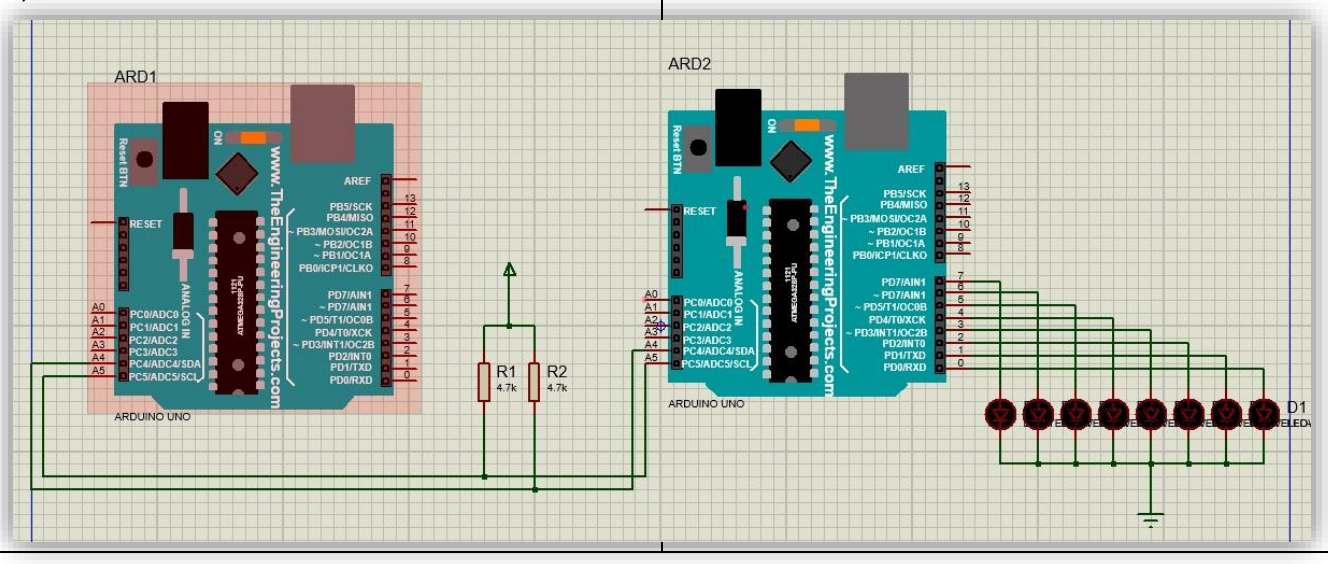
```
/* Master in Write Mode*/

#include<avr/io.h>

void i2c_init (void)
{
TWSR = 0X00; // set prescaler bits to zero
TWBR = 0x47; // SCL frequency is 50K for XTAL = 8Mhz
TWCR = 0x04; // enable TWI module
}

void i2c_start(void)
{
 TWCR = ( 1 << TWINT ) | ( 1 << TWSTA ) | ( 1 << TWEN
);
 while ((TWCR & ( 1 << TWINT ) ) == 0) ;
}

void i2c_write(unsigned char data)
{
 TWDR = data ;
 TWCR = (1<<TWINT) | (1<<TWEN); // enable the
transmission
 while ((TWCR & ( 1 << TWINT ) ) == 0); // polling e.g
checking TWINT flag to check whether START condition
transmitted succesfully
}

void i2c_stop()
{
TWCR = ( 1 <<TWINT )|( 1 << TWEN) | ( 1<< TWSTO);
}

int main(void)
{

i2c_init(); // initialse TWI for master mode
i2c_start(); // transmit start condition
i2c_write(0b11010000); // transmitt Slave address + W0
(SLA+W means to master is writing )
i2c_write(0b10101010); // transmitt data
i2c_stop(); // transmitt start condition
while(1);
return 0;
}
```

## 03. Slave in Read Mode (only one byte) :-

```
/* Slave in Read mode*/

#include<avr/io.h>

void i2c_initslave (unsigned char slaveaddress)
{
TWCR = 0x04;  // enable TWI module by setting TWEN flag
TWAR = slaveaddress;  // set slave address
TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWEA); // init TWI module

}

void i2c_listen()
{
 while ((TWCR & ( 1 << TWINT ) ) == 0); // wait to be addressed by master
}

unsigned char i2c_read(unsigned char last)
{
 if ( last == 0 ) // If want to read more than 1 byte
    TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
 else           // else read only one byte
 TWCR = (1<<TWINT) | (1<<TWEN);
 while ((TWCR & ( 1 << TWINT ) ) == 0);
 return TWDR ;
}

int main(void)
{
DDRD = 0xFF; // PORTD is output
i2c_initslave(0b11010001); // initiate  TWI module as slave with address = 0b11010001
i2c_listen(); // listen to be addressed
PORTD = i2c_read(1); // read only 1 byte
while(1);
return 0;
}
```

## 04. Slave in Write Mode (only one byte) :-

```
/* Slave in write mode*/

#include<avr/io.h>

void i2c_initslave (unsigned char slaveaddress)
{
TWCR = 0x04;  // enable TWI module by setting TWEN flag
TWAR = slaveaddress;  // set slave address
TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWEA); // init TWI module

}

void i2c_listen()
{
 while ((TWCR & ( 1 << TWINT ) ) == 0); // wait to be addressed by master
}

void i2c_send(unsigned char data)
{
TWDR = data ; // copy data to TWDR
TWCR = ( 1 << TWINT ) | ( 1 << TWEN ); // start transmission
 while ((TWCR & ( 1 << TWINT ) ) == 0) ; // wait to complete
}

int main(void)
{
i2c_initslave(0b11010001); // initiate  TWI module as slave with address = 0b11010001
i2c_listen(); // listen to be addressed
i2c_send('U'); //transmitt letter
while(1); // stay here forever
return 0;
}
```



## 05. Master in Read Mode (one byte at a time) :-

```
/* Master in Read Mode*/
#include<avr/io.h>

void i2c_init (void){
TWSR = 0X00; // set prescaler bits to zero
TWBR = 0x47; // SCL frequency is 50K for XTAL = 8Mhz
TWCR = 0x04; // enable TWI module
}

void i2c_start(void){
 TWCR = ( 1 << TWINT ) | ( 1 << TWSTA ) | ( 1 << TWEN );
 while ((TWCR & ( 1 << TWINT ) ) == 0) ;
}

void i2c_write(unsigned char data){
 TWDR = data ;
 TWCR = (1<<TWINT) | (1<<TWEN); // enable the
transmission
 while ((TWCR & ( 1 << TWINT ) ) == 0); // polling e.g
checking TWINT flag to check whether START condition
transmitted succesfully
}

unsigned char i2c_read(unsigned char last){
 if ( last == 0 ) // If want to read more than 1 byte
    TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
 else           // else read only one byte
 TWCR = (1<<TWINT) | (1<<TWEN);
 while ((TWCR & ( 1 << TWINT ) ) == 0);
 return TWDR ;
}

void i2c_stop(){
TWCR = ( 1 <<TWINT )|( 1 << TWEN) | ( 1<< TWSTO);
}

int main(void){
unsigned char i = 0 ;
DDRD = 0xFF; // PORTD is output
i2c_init(); // initialse TWI for master mode

L1:
i2c_start(); // transmit start condition
i2c_write(0b11010001); // transmitt Slave address + R(1)
(SLA+R means to master is reading )
i = i2c_read(0); // 0 for read multiple bytes
PORTD= i; // show byte in PORTD
i2c_stop(); // transmitt stop condition
for(volatile unsigned long i=0; i<100000; i++);
goto L1;

while(1);
return 0;
}
```

## 06. Master in Write Mode (one byte at a time) :-

```
/* Master in Write Mode*/

#include<avr/io.h>

void i2c_init (void){
TWSR = 0X00; // set prescaler bits to zero
TWBR = 0x47; // SCL frequency is 50K for XTAL = 8Mhz
TWCR = 0x04; // enable TWI module
}

void i2c_start(void){
 TWCR = ( 1 << TWINT ) | ( 1 << TWSTA ) | ( 1 << TWEN
);
 while ((TWCR & ( 1 << TWINT ) ) == 0) ;
}

void i2c_write(unsigned char data){
 TWDR = data ;
 TWCR = (1<<TWINT) | (1<<TWEN); // enable the
transmission
 while ((TWCR & ( 1 << TWINT ) ) == 0); // polling e.g
checking TWINT flag to check whether START condition
transmitted succesfully
}

void i2c_stop(){
TWCR = ( 1 <<TWINT )|( 1 << TWEN) | ( 1<< TWSTO);
}

int main(void){
volatile uint8_t shift=0;

i2c_init(); // initialise TWI for master mode

L1:
i2c_start(); // transmit start
i2c_write(0b11010000); // transmitt Slave address +
W0 (SLA+W means to master is writing )
i2c_write(1<<shift); // transmitt data
i2c_stop(); // transmitt start condition
for(volatile long i=0; i<100000; i++);
shift++;
if(shift==8) shift=0;
goto L1;

while(1);
return 0;
}
```

## 07. Slave in Read Mode (one byte at a time) :-

```
/* Slave in Read mode*/

#include<avr/io.h>

void i2c_initslave (unsigned char slaveaddress)
{
TWCR = 0x04;  // enable TWI module by setting TWEN flag
TWAR = slaveaddress;  // set slave address
TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWEA); // init TWI module

}

void i2c_listen()
{
 while ((TWCR & ( 1 << TWINT ) ) == 0); // wait to be addressed by master
}

unsigned char i2c_read(unsigned char last)
{
 if ( last == 0 ) // If want to read more than 1 byte
    TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
 else           // else read only one byte
 TWCR = (1<<TWINT) | (1<<TWEN);
 while ((TWCR & ( 1 << TWINT ) ) == 0);
 return TWDR ;
}

int main(void)
{

DDRD = 0xFF; // PORTD is output
i2c_initslave(0b11010001); // initiate  TWI module as slave with address = 0b11010001

L1:
i2c_listen(); // listen to be addressed
PORTD = i2c_read(0); // 0 for read multiple bytes
goto L1;

while(1);
return 0;
}
```

## 08. Slave in Write Mode (one byte at a time) :-

```
/* Slave in write mode*/

#include<avr/io.h>
volatile uint8_t shift=0;

void i2c_initslave (unsigned char slaveaddress)
{
TWCR = 0x04;  // enable TWI module by setting TWEN flag
TWAR = slaveaddress;  // set slave address
TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWEA); // init TWI module

}

void i2c_listen()
{
 while ((TWCR & ( 1 << TWINT ) ) == 0); // wait to be addressed by master
}

void i2c_send(unsigned char data)
{
TWDR = data ; // copy data to TWDR
TWCR = ( 1 << TWINT ) | ( 1 << TWEN ); // start transmission
 while ((TWCR & ( 1 << TWINT ) ) == 0) ; // wait to complete
}

int main(void)
{

L1:
i2c_initslave(0b11010001); // initiate  TWI module as slave with address = 0b11010001
i2c_listen(); // listen to be addressed
i2c_send(3<<shift); //transmitt letter
shift++;
if(shift==8) shift=0;
goto L1;


while(1); // stay here forever
return 0;
}
```

## 09. Master in Write Mode (Multiple bytes) :-

```
/* Master in Write Mode*/

#include<avr/io.h>

void i2c_init (void){
TWSR = 0X00; // set prescaler bits to zero
TWBR = 0x47; // SCL frequency is 50K for XTAL = 8Mhz
TWCR = 0x04; // enable TWI module
}

void i2c_start(void){
 TWCR = ( 1 << TWINT ) | ( 1 << TWSTA ) | ( 1 << TWEN
);
 while ((TWCR & ( 1 << TWINT ) ) == 0) ;
}

void i2c_write(unsigned char data){
 TWDR = data ;
 TWCR = (1<<TWINT) | (1<<TWEN); // enable the
transmission
 while ((TWCR & ( 1 << TWINT ) ) == 0); // polling e.g
checking TWINT flag to check whether START condition
transmitted succesfully
}

void i2c_stop(){
TWCR = ( 1 <<TWINT )|( 1 << TWEN) | ( 1<< TWSTO);
}

int main(void){
volatile uint8_t shift=0;

i2c_init(); // initialise TWI for master mode

L1:
i2c_start(); // transmit start
i2c_write(0b11010000); // transmitt Slave address +
W0 (SLA+W means to master is writing )

i2c_write(1<<shift); // transmitt data
i2c_write(7<<shift); // transmitt data

i2c_stop(); // transmitt start condition
for(volatile long i=0; i<100000; i++);
shift++;
if(shift==8) shift=0;
goto L1;

while(1);
return 0;
}
```

## 10. Slave in Read Mode (Multiple bytes) :-

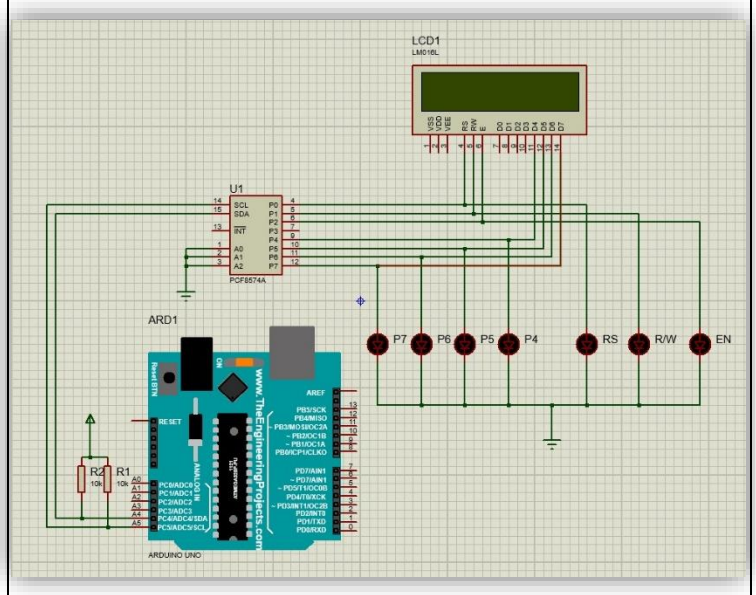
```
/* Slave in Read mode*/

#include<avr/io.h>


void i2c_initslave (unsigned char slaveaddress)
{
TWCR = 0x04;  // enable TWI module by setting TWEN
flag
TWAR = slaveaddress;  // set slave address
TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWEA); // init
TWI module

}


void i2c_listen()
{
 while ((TWCR & ( 1 << TWINT ) ) == 0); // wait to be
addressed by master
}


unsigned char i2c_read(unsigned char last)
{
 if ( last == 0 ) // If want to read more than 1 byte
    TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
 else           // else read only one byte
 TWCR = (1<<TWINT) | (1<<TWEN);
 while ((TWCR & ( 1 << TWINT ) ) == 0);
 return TWDR ;
}

int main(void)
{

DDRD = 0xFF; // PORTD is output
i2c_initslave(0b11010001); // initiate  TWI module as
slave with address = 0b11010001

L1:
i2c_listen(); // listen to be addressed
PORTD = i2c_read(0); // 0 for read multiple bytes
goto L1;

while(1);
return 0;
}
```

## 11. 16*2 LCD in 4 bit mode (Proteus) :-

```
#define command_high 4
#define command_low 0
#define data_high 5
#define data_low 1
#define slave_address 0x70

void i2c_init (void){
TWSR = 0X00; // set prescaler bits to zero
TWBR = 0x47; // SCL frequency is 50K for XTAL = 8Mhz
TWCR = 0x04; // enable TWI module
}

void i2c_start(void){
 TWCR = ( 1 << TWINT ) | ( 1 << TWSTA ) | ( 1 << TWEN );
 while ((TWCR & ( 1 << TWINT ) ) == 0) ;
}

void i2c_write(unsigned char data){
 TWDR = data ;
 TWCR = (1<<TWINT) | (1<<TWEN); // enable the
transmission
 while ((TWCR & ( 1 << TWINT ) ) == 0); // polling e.g
checking TWINT flag to check whether START condition
transmitted succesfully
}

void i2c_stop(){
TWCR = ( 1 <<TWINT )|( 1 << TWEN) | ( 1<< TWSTO);
}

//////// ABOVE ARE THE I2C FUNCTIONS ////////////////

void lcd_init(){
  send_command (0x02);          // 4bit mode
        //send_command (0x28);          // Initialization of
16X2 LCD in 4bit mode
        send_command (0x0E); // Display ON Cursor ON
        send_command (0x06); // Auto Increment cursor
        send_command (0x01); // Clear display
        send_command (0x80); // Cursor at home position
}

void send_command(volatile uint8_t cmd){

        volatile uint8_t high_nibble = cmd & 0xf0;
        volatile uint8_t low_nibble = cmd << 4;

  i2c_start(); // transmit start
  i2c_write(slave_address); // transmitt Slave address + W0
(SLA+W means to master is writing )
  i2c_write(high_nibble + command_high);
  i2c_write(high_nibble + command_low);
  i2c_write(low_nibble + command_high);
        i2c_write(low_nibble + command_low);
  i2c_stop(); // transmitt stop condition
  delayMicroseconds(500);
}
```

```
void send_data(volatile uint8_t data1){

        volatile uint8_t high_nibble = data1 & 0xf0;
        volatile uint8_t low_nibble = data1 << 4;

  i2c_start(); // transmit start
  i2c_write(slave_address); // transmitt Slave address + W0
(SLA+W means to master is writing )
  i2c_write(high_nibble + data_high);
  i2c_write(high_nibble + data_low);
  i2c_write(low_nibble + data_high);
        i2c_write(low_nibble + data_low);
  i2c_stop(); // transmitt stop condition
  delayMicroseconds(50);
}

void lcd_string (char *str)          /* Send string to LCD
function */
{
        int i;
        for(i=0;str[i]!=0;i++)  /* Send each char of string till
the NULL */
        {
                send_data (str[i]);  /* Call LCD data write */
        }
}

int main(void){

  i2c_init(); // initialise TWI for master mode
  lcd_init();  // initialise the 16*2 LCD
  lcd_string("Hello World!");

while(1);
return 0;
}
```

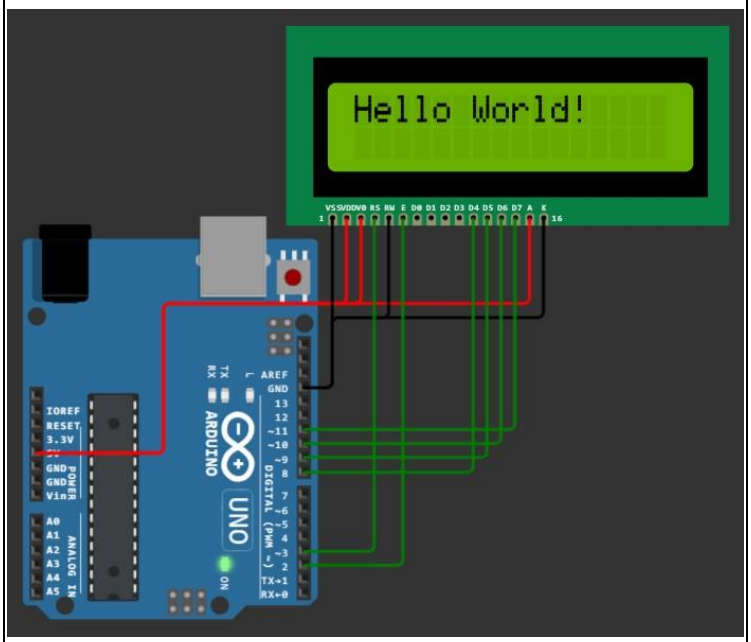
The only difference between wokwi and Proteus for this program is the delay(); function and its value



## 12. 16*2 LCD in 4 bit mode (Wokwi) :-

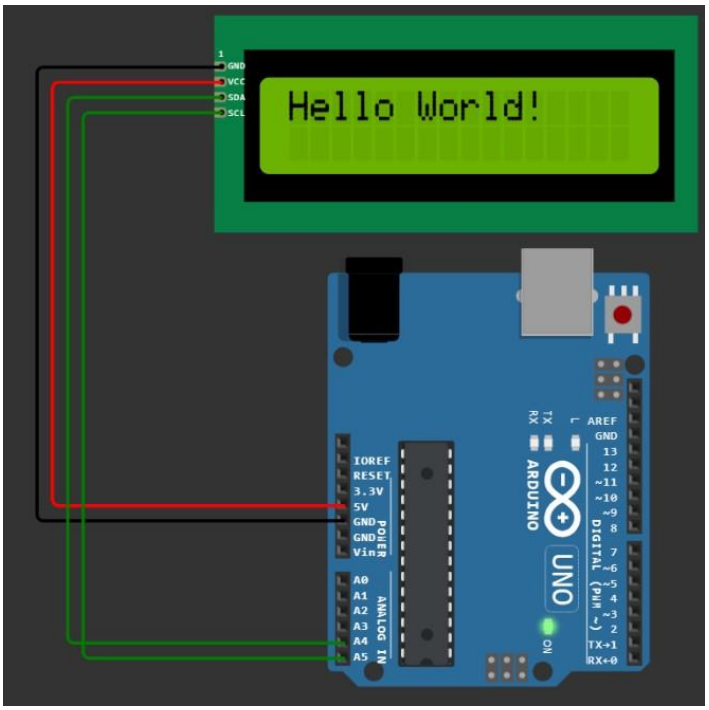
```
/* Note: in this program or any other program does not use the delay() function,if delay()
function is used, then overall program does not work, instead of it, use delayMicrosecond() */


void LCD_Init (void)                /* LCD Initialize function */
{
        delayMicroseconds(2000);                /* LCD Power ON Initialization time >15ms */
        LCD_Command (0x02);  // 4bit mode
        LCD_Command (0x28);  //  4bit mode
        LCD_Command (0x0C);  //Display ON Cursor OFF
        LCD_Command (0x06);  // Auto Increment cursor
        LCD_Command (0x01);  // Clear display
        LCD_Command (0x80);  // Cursor at home position
}



void LCD_Command (volatile uint8_t cmnd)
{
        PORTB = cmnd >> 4;    /* Send upper nibble */
        PORTD = 4;
        delayMicroseconds(200);
        PORTD = 0;
        delayMicroseconds(200);

        PORTB = cmnd & 0x0f;/* Send lower nibble */
        PORTD = 4;
        delayMicroseconds(200);
        PORTD = 0;
        delayMicroseconds(200);
}


void LCD_Char (volatile uint8_t data)
{
        PORTB = data >> 4;    /* Send upper nibble */
        PORTD = 8 + 4;
        delayMicroseconds(200);
        PORTD = 8;
        delayMicroseconds(200);

        PORTB = data & 0x0f;/* Send lower nibble */
        PORTD = 8 + 4;
        delayMicroseconds(200);
        PORTD = 8;
        delayMicroseconds(200);

}
```

```
void LCD_String (volatile uint8_t *str)     /* Send string to LCD function */
{
        int i;
        for(i=0;str[i]!=0;i++)  /* Send each volatile uint8_t of string till the NULL */
        {
                LCD_Char (str[i]);  /* Call LCD data write */
        }
}


int main()
{
  DDRD = 0x0c;
  DDRB = 0x0f;

        LCD_Init();                 /* Initialization of LCD*/
        LCD_String("Hello World!");  /* write string on 1st line of LCD*/

        while(1);                   /* Infinite loop. */
        return 0;
}
```



## 13. 16*2 LCD through I2C Protocol (Wokwi) :-

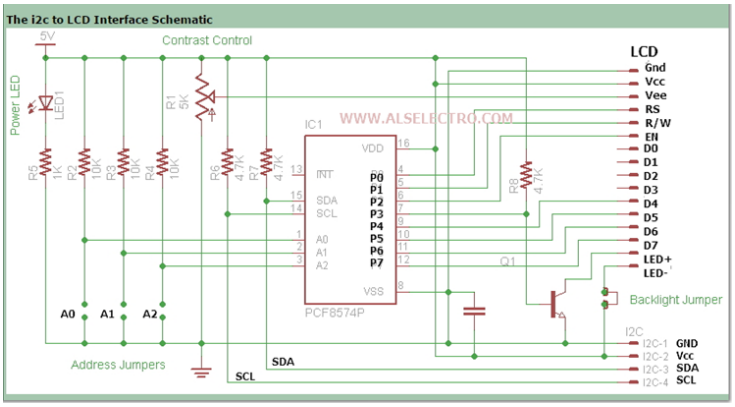
```
#define command_en_high 4 + 8   // RS=0, R/W=0, EN=1 (4), LED+ = 1 (8)
#define command_en_low 0 + 8    // RS=0, R/W=0, EN=0, LED+ = 1 (8)
#define data_en_high 1 + 4 + 8  // RS=1 (1), R/W=0, EN=1 (4), LED+ = 1 (8)
#define data_en_low 1 + 8       // RS=1 (1), R/W=0, EN=0, LED+ = 1 (8)
#define slave_address 0x27 << 1  // slave address of the I2C LCD

void i2c_init (void){
TWSR = 0X00; // set prescaler bits to zero
TWBR = 0x47; // SCL frequency is 50K for XTAL = 8Mhz
TWCR = 0x04; // enable TWI module
}

void i2c_start(void){
 TWCR = ( 1 << TWINT ) | ( 1 << TWSTA ) | ( 1 << TWEN );
 while ((TWCR & ( 1 << TWINT ) ) == 0) ;
}

void i2c_write(unsigned char data){
 TWDR = data ;
 TWCR = (1<<TWINT) | (1<<TWEN); // enable the transmission
 while ((TWCR & ( 1 << TWINT ) ) == 0);
}

void i2c_stop(){
TWCR = ( 1 <<TWINT )|( 1 << TWEN) | ( 1<< TWSTO);
}

////////////// ABOVE ARE THE I2C FUNCTIONS //////////////

////////////// BELOW ARE THE LCD FUNCTIONS //////////////

void lcd_init(){
          send_command (0x02);            // 4bit mode
          //send_command (0x28);          //  4bit mode
          send_command (0x0C); // Display ON Cursor OFF
          send_command (0x06); // Auto Increment cursor
          send_command (0x01); // Clear display
          send_command (0x80); // Cursor at home position
}

void send_command(volatile uint8_t cmd){

          volatile uint8_t high_nibble = cmd & 0xf0;
          volatile uint8_t low_nibble = cmd << 4;

  i2c_start(); // transmit start
  i2c_write(slave_address); // transmitt Slave address + W0 (SLA+W means to master is writing )
  i2c_write(high_nibble + command_en_high);
  i2c_write(high_nibble + command_en_low);
  i2c_write(low_nibble + command_en_high);
          i2c_write(low_nibble + command_en_low);
  i2c_stop(); // transmit stop condition
  delayMicroseconds(50);
}

void send_data(volatile uint8_t data1){

          volatile uint8_t high_nibble = data1 & 0xf0;
          volatile uint8_t low_nibble = data1 << 4;

  i2c_start(); // transmit start
  i2c_write(slave_address); // transmitt Slave address + W0 (SLA+W means to master is writing )
  i2c_write(high_nibble + data_en_high);
  i2c_write(high_nibble + data_en_low);
  i2c_write(low_nibble + data_en_high);
  i2c_write(low_nibble + data_en_low);
  i2c_stop(); // transmitt stop condition
  delayMicroseconds(50);
}

void lcd_string (char *str){
          int i;
          for(i=0;str[i]!=0;i++)
          send_data (str[i]);  /* Call LCD data write */
}

int main(void){

  i2c_init(); // initialise TWI for master mode
  lcd_init();  // initialise the 16*2 LCD
  lcd_string("Hello World!");

while(1);
return 0;
}
```

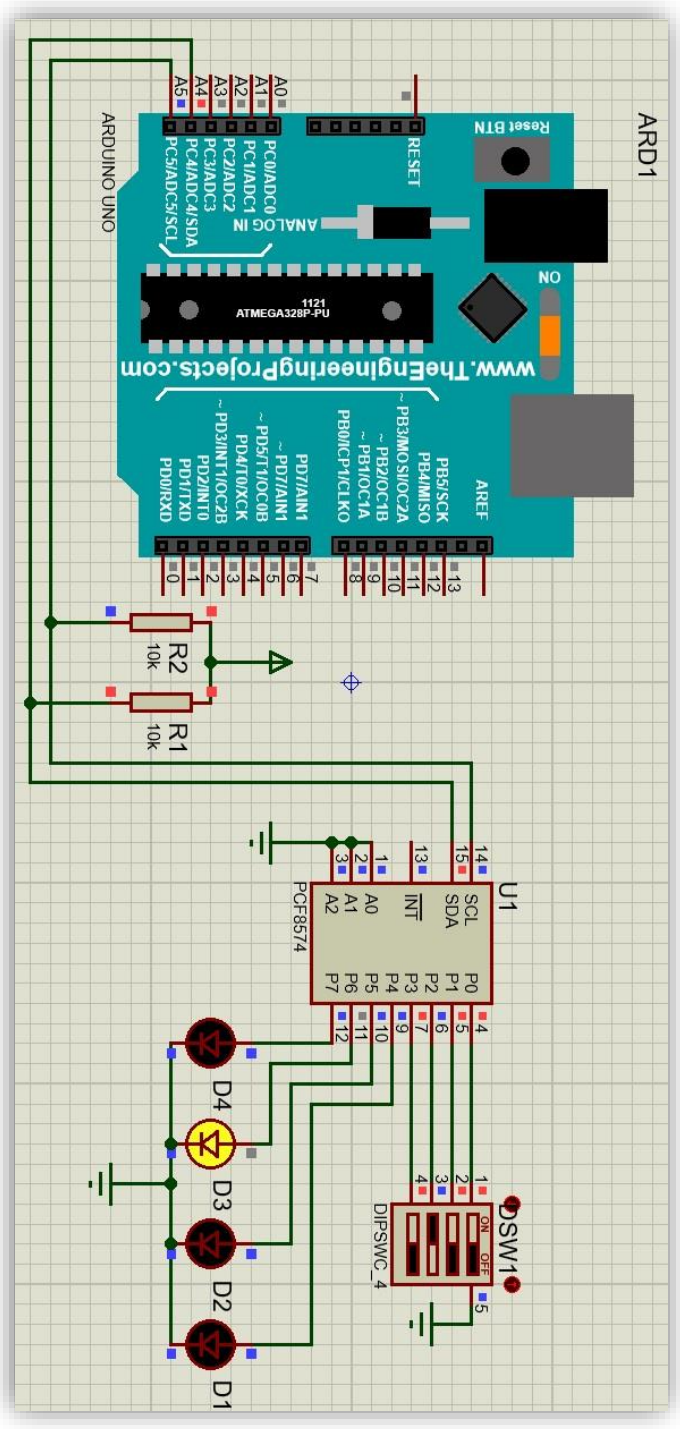


## 14. I2C Read & Write Program :-

```
#include<avr/io.h>

void i2c_init (void){
TWSR = 0X00; // set prescaler bits to zero
TWBR = 0x47; // SCL frequency is 50K for XTAL = 8Mhz
TWCR = 0x04; // enable TWI module
}

void i2c_start(void){
 TWCR = ( 1 << TWINT ) | ( 1 << TWSTA ) | ( 1 << TWEN );
 while ((TWCR & ( 1 << TWINT ) ) == 0) ;
}

void i2c_write(unsigned char data){
 TWDR = data ;
 TWCR = (1<<TWINT) | (1<<TWEN); // enable the
transmission
 while ((TWCR & ( 1 << TWINT ) ) == 0);
}

unsigned char i2c_read(unsigned char last){
 if ( last == 0 ) // If want to read more than 1 byte
    TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
 else           // else read only one byte
 TWCR = (1<<TWINT) | (1<<TWEN);
 while ((TWCR & ( 1 << TWINT ) ) == 0);
 return TWDR ;
}

void i2c_stop(){
TWCR = ( 1 <<TWINT )|( 1 << TWEN) | ( 1<< TWSTO);
}

int main(void){
  volatile uint8_t rcvData=0;

   while(1){

     i2c_start();
     i2c_write(0x40);                // slave address + write
     i2c_write(rcvData|0x0F);   // Write lower nibble to HIGH
and higher nibble to rcvData
     i2c_stop();

     i2c_start();
     i2c_write(0x41);          // slave address + read
     rcvData=i2c_read(1);   // Read the input of lower nibble
     i2c_stop();

     rcvData <<= 4;         //Shift the inputs right to the output
     rcvData = ~rcvData;

     delayMicroseconds(100);
   }
   return 0;
}
```

Below figure only for the I2C LCD



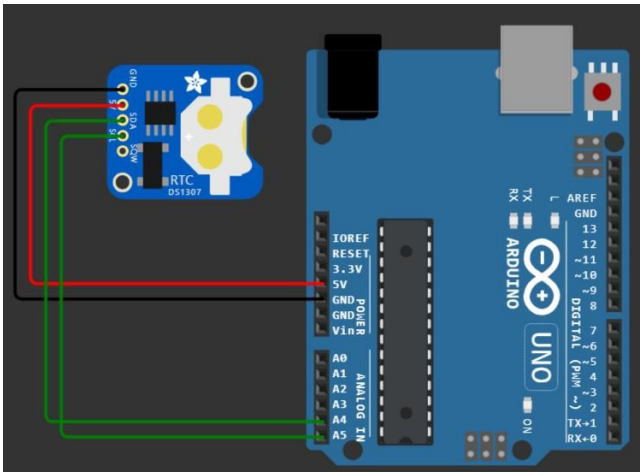



## 15. I2C RTC Program (Serial Print) :-

```
#include<avr/io.h>        /* Page - 1 */

void i2c_init (void){
TWSR = 0X00; // set prescaler bits to zero
TWBR = 0x47; // SCL frequency is 50K for XTAL = 8Mhz
TWCR = 0x04; // enable TWI module
}

void i2c_start(void){
TWCR = ( 1 << TWINT ) | ( 1 << TWSTA ) | ( 1 << TWEN
);
 while ((TWCR & ( 1 << TWINT ) ) == 0) ;
}

void i2c_write(unsigned char data){
 TWDR = data ;
 TWCR = (1<<TWINT) | (1<<TWEN); // enable the
transmission
 while ((TWCR & ( 1 << TWINT ) ) == 0);
}

unsigned char i2c_read(unsigned char last){
 if ( last == 0 ) // If want to read more than 1 byte
    TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
 else           // else read only one byte
 TWCR = (1<<TWINT) | (1<<TWEN);
 while ((TWCR & ( 1 << TWINT ) ) == 0);
 return TWDR ;
}

void i2c_stop(){
TWCR = ( 1 <<TWINT )|( 1 << TWEN) | ( 1<< TWSTO);
}

//-----------------------------------------------------

volatile uint8_t read_second(){

  volatile uint8_t sec, second_ones, second_tens;
  i2c_start();
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x00);       // second address 0x00
  i2c_start();           // repeated start
  i2c_write(0b11010001);  // slave address + read
  sec = i2c_read(1);
  i2c_stop();

  second_ones = 0x0f & sec;
  second_tens = 0x07 & (sec >> 4);
  sec = (second_tens * 10) + second_ones;   //
calculating the atual second value
  return sec;
}
```

```
volatile uint8_t read_minute(){     /* Page - 2 */

  volatile uint8_t minute, minute_ones, minute_tens;
  i2c_start();
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x01);       // minute address 0x01
  i2c_start();           // repeated start
  i2c_write(0b11010001);  // slave address + read
  minute = i2c_read(1);
  i2c_stop();

  minute_ones = 0x0f & minute;
  minute_tens = minute >> 4;
  minute = (minute_tens * 10) + minute_ones;   //
calculating the atual minute value
  return minute;
}

volatile uint8_t read_hour(){

  volatile uint8_t hour, hour_ones, hour_tens;
  i2c_start();
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x02);       // hour address 0x02
  i2c_start();           // repeated start
  i2c_write(0b11010001);  // slave address + read
  hour = i2c_read(1);
  i2c_stop();

  hour_ones = 0x0f & hour;
  hour_tens = 0x03 & (hour >> 4);
  hour = (hour_tens * 10) + hour_ones;   // calculating
the atual hour value
  return hour;
}

volatile uint8_t read_day(){

  volatile uint8_t day;
  i2c_start();
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x03);       // day address 0x03
  i2c_start();           // repeated start
  i2c_write(0b11010001);  // slave address + read
  day = i2c_read(1);
  i2c_stop();

  day &= 0x07;                    // calculating the atual
day value
  return day;
}

volatile uint8_t read_date(){

  volatile uint8_t date, date_ones, date_tens;
```

```
  i2c_start();      /* Page - 3 */
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x04);       // date address 0x04
  i2c_start();          // repeated start
  i2c_write(0b11010001);  // slave address + read
  date = i2c_read(1);
  i2c_stop();

  date_ones = 0x0f & date;
  date_tens = 0x03 & (date >> 4);
  date = (date_tens * 10) + date_ones;   // calculating
the atual date value
  return date;
}

volatile uint8_t read_month(){

  volatile uint8_t month, month_ones, month_tens;
  i2c_start();
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x05);       // month address 0x05
  i2c_start();          // repeated start
  i2c_write(0b11010001);  // slave address + read
  month = i2c_read(1);
  i2c_stop();

  month_ones = 0x0f & month;
  month_tens = 0x01 & (month >> 4);
  month = (month_tens * 10) + month_ones;   //
calculating the atual month value
  return month;
}

volatile uint8_t read_year(){

  volatile uint8_t year, year_ones, year_tens;
  i2c_start();
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x06);       // year address 0x06
  i2c_start();          // repeated start
  i2c_write(0b11010001);  // slave address + read
  year = i2c_read(1);
  i2c_stop();

  year_ones = 0x0f & year;
  year_tens = year >> 4;
  year = (year_tens * 10) + year_ones;   // calculating
the atual year value
  return year;
}

int main(){
  Serial.begin(9600);
  volatile uint8_t seconds, minutes, hours, days, dates,
months, years;
  char day[][12] = {"Saturday", "Sunday", "Monday",
"Tuesday", "Wednessday", "Thursday", "Friday"};

  i2c_init();          /* Page - 4 */

  while(1){

  Serial.print("TIME:- ");     // printing the time //
  hours = read_hour();
  Serial.print(hours);
  Serial.print(':');
  minutes = read_minute();
  Serial.print(minutes);
  Serial.print(':');
  seconds = read_second();
  Serial.print(seconds);
  Serial.println();


  Serial.print("DATE:- ");     // printing the date //
  dates = read_date();
  Serial.print(dates);
  Serial.print('/');
  months = read_month();
  Serial.print(months);
  Serial.print('/');
  years = read_year();
  Serial.print("20");
  Serial.print(years);
  Serial.println();


  Serial.print("DAY:- ");       // printing the day //
  days = read_day();
  Serial.print(day[days]);

  Serial.println();
  Serial.println();
  Serial.println();
  for(volatile long i=0; i<100000; i++);  // delay //
  }
  return 0;
}
```

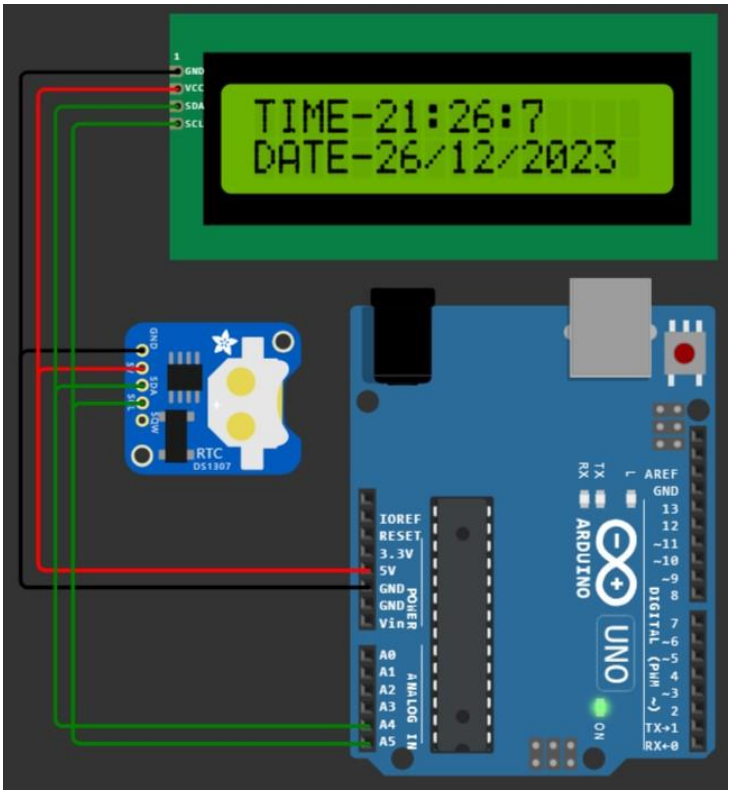


## 16. I2C RTC with I2C 16*2 LCD Program :-

```
#include "I2C.h"                          // Page 1 //
#include "LCD1602.h"
#include "RTC1307.h"

int main(){

  volatile uint8_t seconds, minutes, hours, days, dates,
months, years;
  char day[][12] = {" Sunday", " Monday", " Tuesday", "
Wednessday", " Thursday", " Friday", " Saturday",};

  i2c_init();
  lcd_init();

while(1){

  for(volatile long i=0; i<300; i++){    // printing the Time &
Date for a particular time //

  lcd_cursor(0,0);
  lcd_string("TIME-");
  hours = read_hour();  // for reading the hour
  lcd_number(hours);    // for printing the hour
  lcd_string(":");
  minutes = read_minute();  // for reading the minute
  lcd_number(minutes);      // for printing the minute
  lcd_string(":");
  seconds = read_second();  // for reading the second
  lcd_number(seconds);      // for printing the second
  lcd_string("     ");

  lcd_cursor(1,0);
  lcd_string("DATE-");
  dates = read_date();  // for reading the date
  lcd_number(dates);    // for printing the date
  lcd_string("/");
  months = read_month();  // for reading the month
  lcd_number(months);      // for printing the month
  lcd_string("/");
  years = read_year();   // for reading the year
  lcd_number(20);
  lcd_number(years);      // for printing the year
  lcd_string("     ");
  }

  lcd_clear();
  lcd_cursor(0,0);
  lcd_string("Today is:-");
  lcd_cursor(1,0);
  days = read_day();       // for reading the day
  lcd_string(day[days - 1]);    // for printing the day
  lcd_string("     ");
  for(volatile long i=0; i<1000000; i++);  // delay //
  }
  return 0;
}
```

## # I2C.h (header file) #

```
#include<avr/io.h>                    // Page 2 //

void i2c_init (void){
TWSR = 0X00; // set prescaler bits to zero
TWBR = 0x47; // SCL frequency is 50K for XTAL = 8Mhz
TWCR = 0x04; // enable TWI module
}

void i2c_start(void){
 TWCR = ( 1 << TWINT ) | ( 1 << TWSTA ) | ( 1 << TWEN );
 while ((TWCR & ( 1 << TWINT ) ) == 0) ;
}

void i2c_write(unsigned char data){
 TWDR = data ;
 TWCR = (1<<TWINT) | (1<<TWEN); // enable the
transmission
 while ((TWCR & ( 1 << TWINT ) ) == 0);
}

unsigned char i2c_read(unsigned char last){
 if ( last == 0 ) // If want to read more than 1 byte
    TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
 else           // else read only one byte
 TWCR = (1<<TWINT) | (1<<TWEN);
 while ((TWCR & ( 1 << TWINT ) ) == 0);
 return TWDR ;
}

void i2c_stop(){
TWCR = ( 1 <<TWINT )|( 1 << TWEN) | ( 1<< TWSTO);
}
```



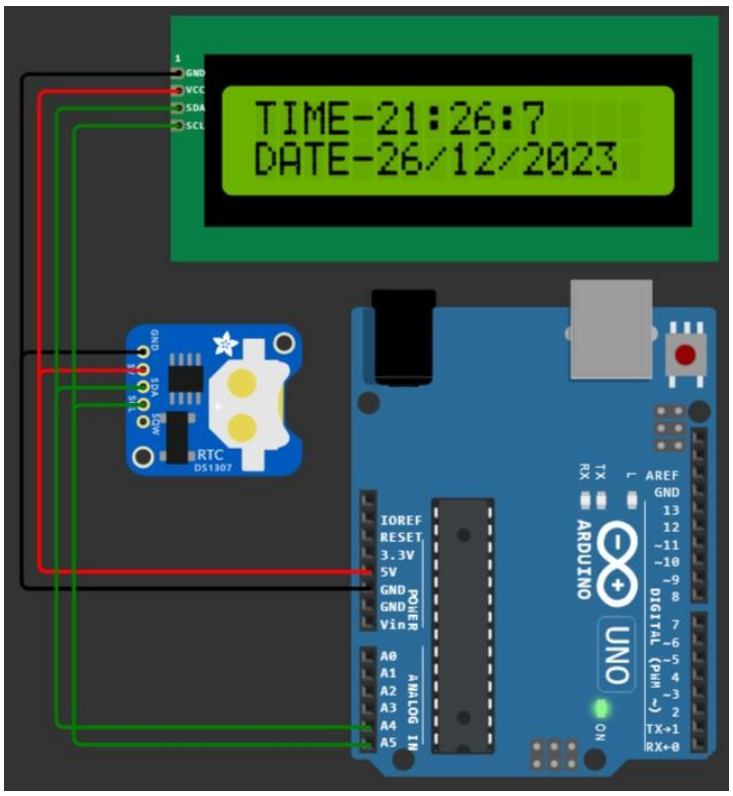
## # RTC1307.h (header file) #

```
volatile uint8_t read_second(){                    // Page 3 //
  volatile uint8_t sec, second_ones, second_tens;
  i2c_start();
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x00);        // second address 0x00
  i2c_start();            // repeated start
  i2c_write(0b11010001);  // slave address + read
  sec = i2c_read(1);
  i2c_stop();

  second_ones = 0x0f & sec;
  second_tens = 0x07 & (sec >> 4);
  sec = (second_tens * 10) + second_ones;   // calculating the
actual second value
  return sec;
}

volatile uint8_t read_minute(){
  volatile uint8_t minute, minute_ones, minute_tens;
  i2c_start();
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x01);        // minute address 0x01
  i2c_start();            // repeated start
  i2c_write(0b11010001);  // slave address + read
  minute = i2c_read(1);
  i2c_stop();

  minute_ones = 0x0f & minute;
  minute_tens = minute >> 4;
  minute = (minute_tens * 10) + minute_ones;   // calculating
the actual minute value
  return minute;
}

volatile uint8_t read_hour(){
  volatile uint8_t hour, hour_ones, hour_tens;
  i2c_start();
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x02);        // hour address 0x02
  i2c_start();            // repeated start
  i2c_write(0b11010001);  // slave address + read
  hour = i2c_read(1);
  i2c_stop();

  hour_ones = 0x0f & hour;
  hour_tens = 0x03 & (hour >> 4);
  hour = (hour_tens * 10) + hour_ones;   // calculating the
actual hour value
  return hour;
}

volatile uint8_t read_day(){
  volatile uint8_t day;
  i2c_start();
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x03);        // day address 0x03
  i2c_start();            // repeated start
  i2c_write(0b11010001);  // slave address + read
  day = i2c_read(1);
  i2c_stop();                                      // Page 4 //

  day &= 0x07;                    // calculating the actual day
value
  return day;
}

volatile uint8_t read_date(){
  volatile uint8_t date, date_ones, date_tens;
  i2c_start();
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x04);        // date address 0x04
  i2c_start();            // repeated start
  i2c_write(0b11010001);  // slave address + read
  date = i2c_read(1);
  i2c_stop();

  date_ones = 0x0f & date;
  date_tens = 0x03 & (date >> 4);
  date = (date_tens * 10) + date_ones;   // calculating the
actual date value
  return date;
}

volatile uint8_t read_month(){
  volatile uint8_t month, month_ones, month_tens;
  i2c_start();
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x05);        // month address 0x05
  i2c_start();            // repeated start
  i2c_write(0b11010001);  // slave address + read
  month = i2c_read(1);
  i2c_stop();

  month_ones = 0x0f & month;
  month_tens = 0x01 & (month >> 4);
  month = (month_tens * 10) + month_ones;   // calculating
the actual month value
  return month;
}

volatile uint8_t read_year(){
  volatile uint8_t year, year_ones, year_tens;
  i2c_start();
  i2c_write(0b11010000);  // slave address + write
  i2c_write(0x06);        // year address 0x06
  i2c_start();            // repeated start
  i2c_write(0b11010001);  // slave address + read
  year = i2c_read(1);
  i2c_stop();

  year_ones = 0x0f & year;
  year_tens = year >> 4;
  year = (year_tens * 10) + year_ones;   // calculating the
actual year value
  return year;
}
```

# # lcd1602.h (header file) #

```
void send_command(volatile uint8_t cmd);    // Page 5 //

#define command_en_high 4 + 8   // RS=0, R/W=0, EN=1 (4),
LED+ = 1 (8)
#define command_en_low 0 + 8    // RS=0, R/W=0, EN=0,
LED+ = 1 (8)
#define data_en_high 1 + 4 + 8  // RS=1 (1), R/W=0, EN=1
(4), LED+ = 1 (8)
#define data_en_low 1 + 8       // RS=1 (1), R/W=0, EN=0,
LED+ = 1 (8)
#define slave_address 0x27 << 1  // slave address of the I2C
LCD

void lcd_init(){
  send_command (0x02);                    // 4bit mode
          //send_command (0x28);          //  4bit mode
          send_command (0x0C); // Display ON Cursor OFF
          send_command (0x06); // Auto Increment cursor
          send_command (0x01); // Clear display
          send_command (0x80); // Cursor at home position
}

void send_command(volatile uint8_t cmd){

          volatile uint8_t high_nibble = cmd & 0xf0;
          volatile uint8_t low_nibble = cmd << 4;

  i2c_start(); // transmit start
  i2c_write(slave_address); // transmitt Slave address + W0
(SLA+W means to master is writing )

i2c_write(high_nibble + command_en_high);
  i2c_write(high_nibble + command_en_low);
  i2c_write(low_nibble + command_en_high);
          i2c_write(low_nibble + command_en_low);
  i2c_stop(); // transmit stop condition
  delayMicroseconds(50);
}

void send_data(volatile uint8_t data1){

          volatile uint8_t high_nibble = data1 & 0xf0;
          volatile uint8_t low_nibble = data1 << 4;

  i2c_start(); // transmit start
  i2c_write(slave_address); // transmitt Slave address + W0
(SLA+W means to master is writing )
  i2c_write(high_nibble + data_en_high);
  i2c_write(high_nibble + data_en_low);
  i2c_write(low_nibble + data_en_high);
  i2c_write(low_nibble + data_en_low);
  i2c_stop(); // transmitt stop condition
  delayMicroseconds(50);
}
```

```
void lcd_string (char *str){                    // Page 6 //
          int i;
          for(i=0;str[i]!=0;i++)
          send_data (str[i]);  /* Call LCD data write */
}

void lcd_number(volatile uint16_t number){
  volatile char i=0, num[8]={0};

  while(number){
    num[i++] = number%10;
    number/=10;
  }
  i--;
  for(; i>=0; i--)
  send_data(num[i] + 48);
}

void lcd_cursor(volatile uint8_t row, volatile uint8_t
column){
  if(row==0) send_command(0x80 + column);
  else if(row==1) send_command(0xc0 + column);
}

void lcd_clear(){
  send_command(0x01);
}
```



# -: SPI Protocol :-

## 01. SPI General Program (Master) :-

[CPOL=1, CPHA=1]

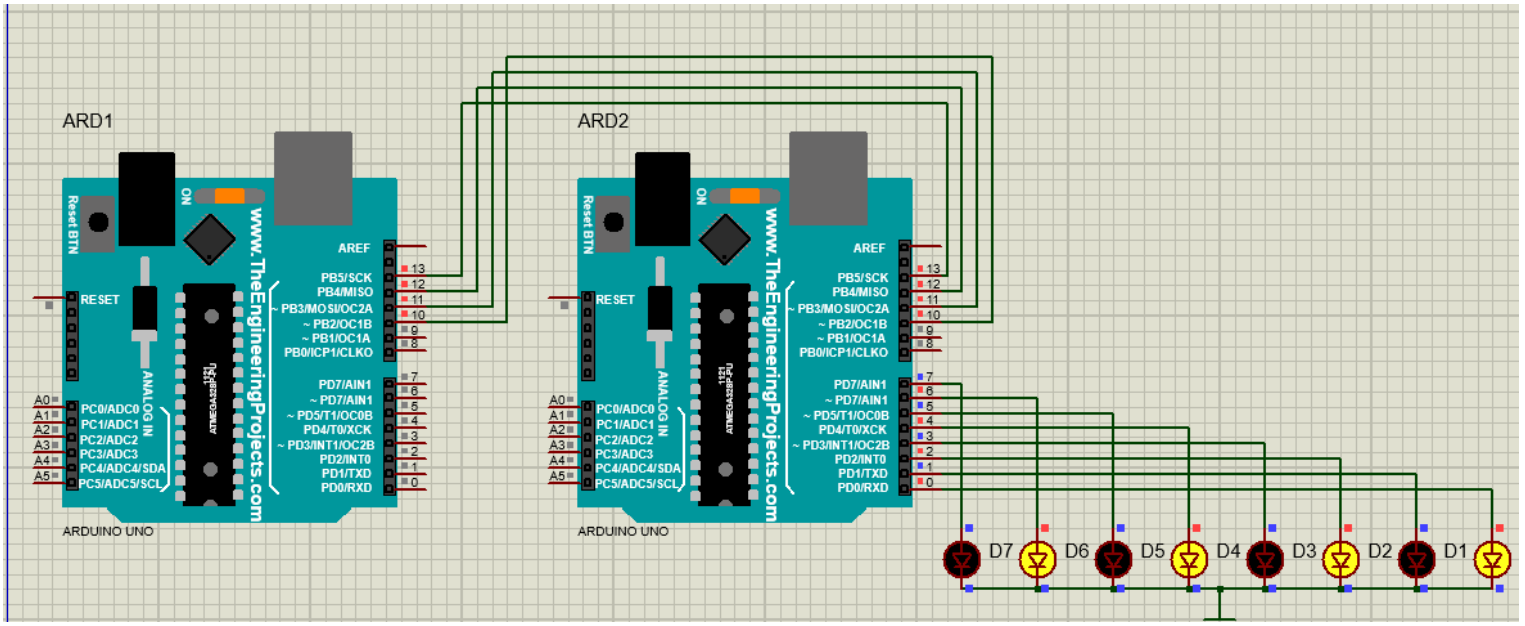
```
void SPI_MasterInit(void){
  DDRB = 0x2c;  // SCK, MOSI, SS set as Output & MISO as Input
  PORTB |= 0x04; // Make SS pin High
  SPCR |= (1<<SPE) | (1<<MSTR) | (1<<CPOL) | (1<<CPHA) | (1<<SPR0);
}

void SPI_MasterTransmit(volatile uint8_t cData){
  SPDR = cData;
  PORTB &= ~0x04;              // Make SS pin Low
  while(!(SPSR & (1<<SPIF)));    // Wait until the Serial Transfer is completed
  PORTB |= 0x04;               // Make SS pin High
  delayMicroseconds(2);
}

void setup(){
  SPI_MasterInit();
  SPI_MasterTransmit(0x55);
}

void loop(){}
```

## (Slave) :-

```
void SPI_SlaveInit(void)
{
DDRB = 0x10;  // MISO set as Output & SCK, MOSI, SS as Input
SPCR |= (1<<SPE) | (1<<CPOL) | (1<<CPHA);
}

char SPI_SlaveReceive(void)
{
while(!(SPSR & (1<<SPIF)));   // Wait until the Serial Transfer is completed
return SPDR;
}

void setup(){
  DDRD = 0xff;   // Set Port D as Output
  SPI_SlaveInit();
  PORTD = SPI_SlaveReceive();
}

void loop(){}
```

Only Master to Slave Transmission

(Half Duplex)



## 02. SPI General Program (Master) :-

[CPOL=0, CPHA=0]

```
void SPI_MasterInit(void){
 DDRB = 0x2c;  // SCK, MOSI, SS set as Output & MISO as Input
 PORTB |= 0x04; // Make SS pin High
 SPCR |= (1<<SPE) | (1<<MSTR) | (1<<SPR0);
}

void SPI_MasterTransmit(volatile uint8_t cData){
 SPDR = cData;
 PORTB &= ~0x04;              // Make SS pin Low
 while(!(SPSR & (1<<SPIF)));   // Wait until the Serial Transfer is completed
 PORTB |= 0x04;              // Make SS pin High
 delayMicroseconds(2);
}

void setup(){
 SPI_MasterInit();
 SPI_MasterTransmit(0x55);
}

void loop(){}
```

## (Slave) :-

```
void SPI_SlaveInit(void){
DDRB = 0x10;  // MISO set as Output & SCK, MOSI, SS as Input
SPCR |= (1<<SPE);
}

char SPI_SlaveReceive(void){
while(!(SPSR & (1<<SPIF)));   // Wait until the Serial Transfer is completed
return SPDR;
}

void setup(){
 DDRD = 0xff;   // Set Port D as Output
 SPI_SlaveInit();
 PORTD = SPI_SlaveReceive();
}

void loop(){}
```

## 03. SPI LED Shift Program (Master) :-

```
void SPI_MasterInit(void){
 DDRB = 0x2c;  // SCK, MOSI, SS set as Output
 PORTB |= 0x04; // Make SS pin High
 SPCR |= (1<<SPE) | (1<<MSTR) | (1<<SPR0);
}

void SPI_MasterTransmit(volatile uint8_t cData){
 SPDR = cData;
 PORTB &= ~0x04;              // Make SS pin Low
 while(!(SPSR & (1<<SPIF)));
 PORTB |= 0x04;              // Make SS pin High
 delayMicroseconds(2);
}

void setup(){
 volatile uint8_t shift=0;
 SPI_MasterInit();
 while(1){
 SPI_MasterTransmit(3<<shift);
 shift++;
 if(shift==8) shift=0;
 for(volatile long i=0; i<100000; i++);
 }}

void loop(){}
```

## (Slave) :-

```
void SPI_SlaveInit(void){
DDRB = 0x10;  // MISO set as Output & SCK, MOSI, SS as Input
SPCR |= (1<<SPE);
}

char SPI_SlaveReceive(void){
while(!(SPSR & (1<<SPIF)));   // Wait until the Serial Transfer is completed
return SPDR;
}

void setup(){
 DDRD = 0xff;   // Set Port D as Output
 SPI_SlaveInit();
 while(1) PORTD = SPI_SlaveReceive();
}

void loop(){}
```

## 04. SPI LED Program [Full Duplex] (Master) :-

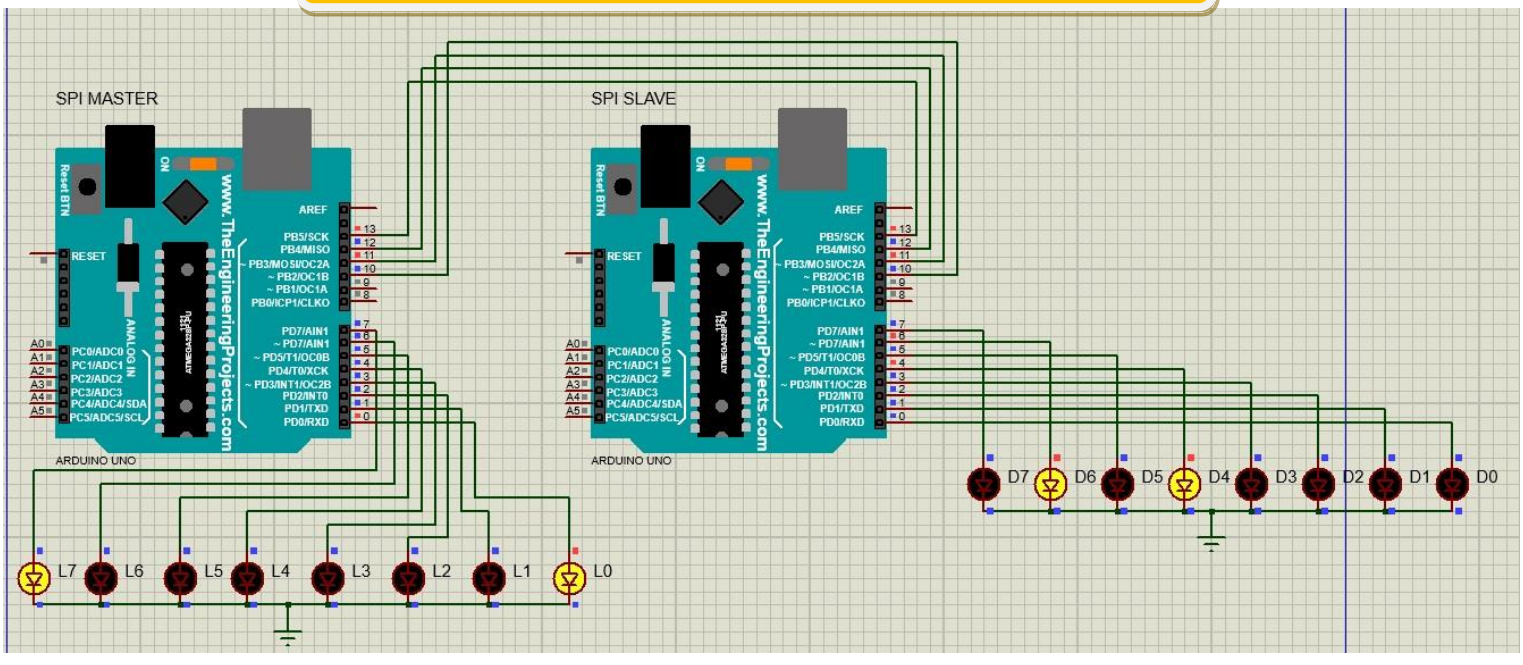
```
void SPI_MasterInit(void){
  DDRB = 0x2c;  // SCK, MOSI, SS set as Output & MISO as Input
  PORTB |= 0x04; // Make SS pin High
  SPCR |= (1<<SPE) | (1<<MSTR) | (1<<SPR0);
}

void SPI_MasterTransmit(volatile uint8_t cData){
  SPDR = cData;
  PORTB &= ~0x04;              // Make SS pin Low
  while(!(SPSR & (1<<SPIF)));    // Wait until the Serial Transfer is completed
  PORTB |= 0x04;               // Make SS pin High
}

char SPI_MasterReceive(void){
  while(!(SPSR & (1<<SPIF)));   // Wait until the Serial Transfer is completed
  return SPDR;
}

void setup(){
  DDRD = 0xff;
  SPI_MasterInit();

  while(1){
  SPI_MasterTransmit(0x50);
  PORTD = SPI_MasterReceive();
  }
}
void loop(){}
```

## (Slave) :-

```
void SPI_SlaveInit(void){
DDRB = 0x10;  // MISO set as Output & SCK, MOSI, SS as Input
SPCR |= (1<<SPE);
}

char SPI_SlaveReceive(void){
while(!(SPSR & (1<<SPIF)));   // Wait until the Serial Transfer is completed
return SPDR;
}

void SPI_SlaveTransmit(volatile uint8_t cData){
  SPDR = cData;
  while(!(SPSR & (1<<SPIF)));    // Wait until the Serial Transfer is completed
  //delayMicroseconds(2);
}

void setup(){
  DDRD = 0xff;   // Set Port D as Output
  SPI_SlaveInit();

  while(1) {
    PORTD = SPI_SlaveReceive();
    SPI_SlaveTransmit(0x03);
  }
}

void loop(){}
```

In real hardware, this program works fine but in Proteus, the Master receives wrong data, Ex – The Slave send data 0x03 but the Master receives 0x81 as shown in the figure.

### Master to Slave & Vice-Versa (Full Duplex)



## 05. SPI Keyboard & LCD Program (Master) :-

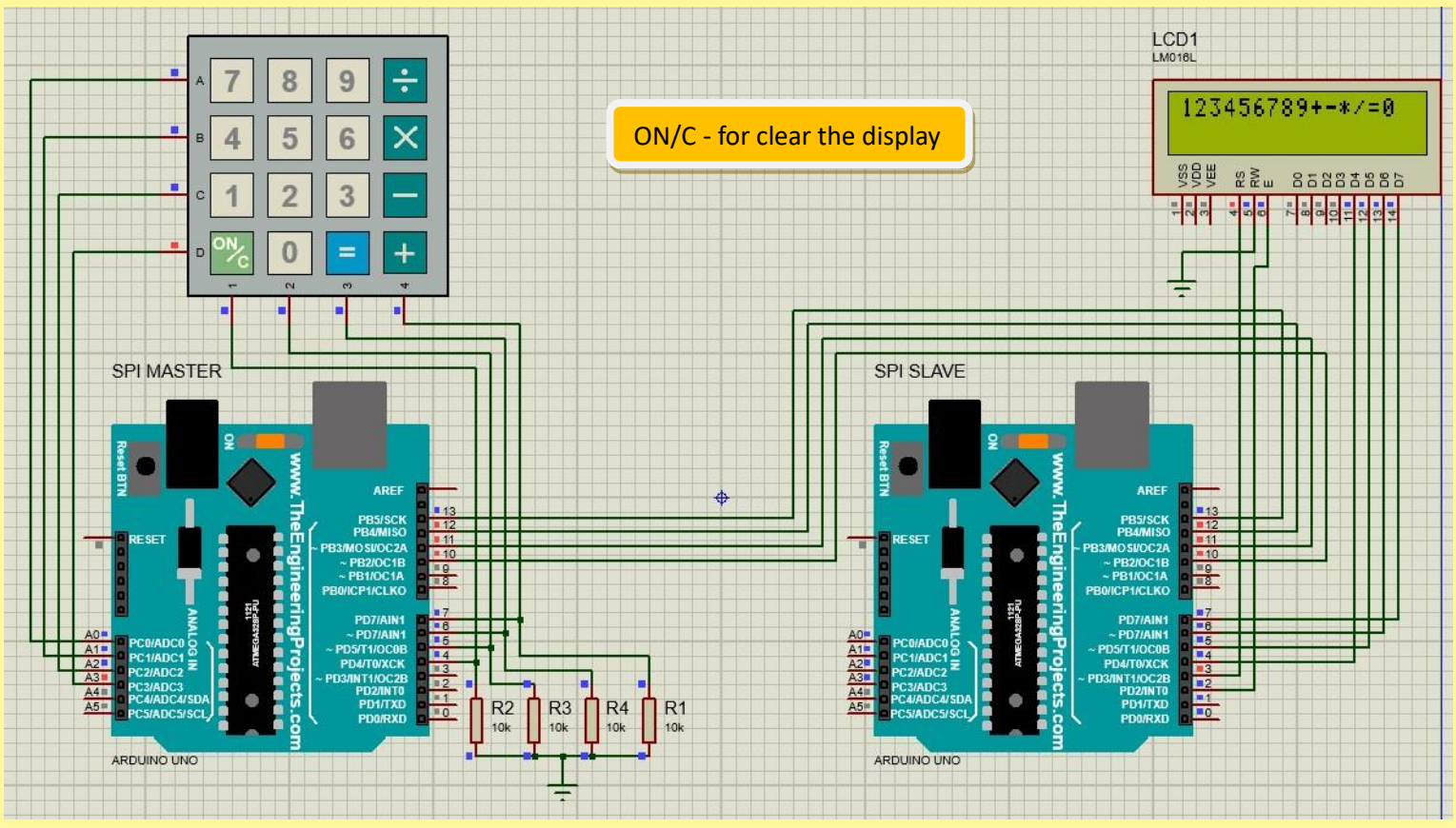
```
//---------------------------------- SPI Functions ----------------------------------//

void SPI_MasterInit(void){
 DDRB = 0x2c;  // SCK, MOSI, SS set as Output & MISO as Input
 PORTB |= 0x04; // Make SS pin High
 SPCR |= (1<<SPE) | (1<<MSTR) | (1<<SPR0);
}


void SPI_MasterTransmit(volatile uint8_t cData){
 SPDR = cData;
 PORTB &= ~0x04;              // Make SS pin Low
 while(!(SPSR & (1<<SPIF)));    // Wait until the Serial Transfer is completed
 PORTB |= 0x04;               // Make SS pin High
}


char SPI_MasterReceive(void){
 while(!(SPSR & (1<<SPIF)));   // Wait until the Serial Transfer is completed
 return SPDR;
}

 //-------------------- KEYBOARD SCANNING FUNCTION ----------------------//

volatile char checkKey(){
  volatile char row, col, key;
  for(row=0; row<4; row++){
   PORTC = 1 << row;
   if(PIND){ col=(PIND) >> 4; break;}
}

 if(row==0){
  if(col==1) key='7';
  else if(col==2) key='8';
  else if(col==4) key='9';
  else if(col==8) key='/';
 }
 else if(row==1){
  if(col==1) key='4';
  else if(col==2) key='5';
  else if(col==4) key='6';
  else if(col==8) key='*';
 }
 else if(row==2){
  if(col==1) key='1';
  else if(col==2) key='2';
  else if(col==4) key='3';
  else if(col==8) key='-';
 }
```

## (Slave) :-

```
//------------------------------ SPI Functions ----------------------------//

void SPI_SlaveInit(void){
DDRB = 0x10;  // MISO set as Output & SCK, MOSI, SS as Input
SPCR |= (1<<SPE);
}

char SPI_SlaveReceive(void){
while(!(SPSR & (1<<SPIF)));   // Wait until the Serial Transfer is completed
return SPDR;
}

//------------------------------ LCD Functions ----------------------------//

void LCD_Init (void)
{
 LCD_Command (0x02);// 8bit mode
        LCD_Command (0x28);  // 8bit mode
        LCD_Command (0x0C);  //Display ON Cursor OFF
        LCD_Command (0x06);  // Auto Increment cursor
        LCD_Command (0x01);  // Clear display
        LCD_Command (0x80);  // Cursor at home position
 delayMicroseconds(2000);
}

void LCD_Command (volatile uint8_t cmnd)
{
        PORTD = (cmnd & 0xf0) + 4;    /* Send upper nibble */
        delayMicroseconds(50);
        PORTD &= ~4;
        delayMicroseconds(50);

        PORTD = (cmnd << 4) + 4;    /* Send lower nibble */
        delayMicroseconds(50);
        PORTD &= ~4;
        delayMicroseconds(50);
}

void LCD_Data (volatile uint8_t data)
{
        PORTD = (data & 0xf0) + 12;  /* Send upper nibble */
        delayMicroseconds(50);
        PORTD &= ~4;
        delayMicroseconds(50);

        PORTD = (data << 4) + 12;    /* Send lower nibble */
        delayMicroseconds(50);
        PORTD &= ~4;
        delayMicroseconds(50);
}
```

```
else if(row==3){
   if(col==1) key='c';
   else if(col==2) key='0';
   else if(col==4) key='=';
   else if(col==8) key='+';
  }
  else key=0;
  return key;
}

//---------------------------------- Main Function ----------------------
------//

void setup(){
  DDRD = 0;
  DDRC = 0x0F;
  volatile char send_data;

  SPI_MasterInit();
  while(1){
    send_data = checkKey();
    if(send_data) SPI_MasterTransmit(send_data);
    for(volatile long i=0; i<70000; i++);
  }
}
```

```
//------------------------------ Main Function ---------------------------
-//

void setup(){
  DDRD = 0xff;
  DDRC = 0x0f;
  volatile uint8_t data;

  SPI_SlaveInit();
          LCD_Init();                  /* Initialization of LCD*/

          while(1){
    data = SPI_SlaveReceive();
    if(data=='c') LCD_Command(0x01);  // clear the display
    else LCD_Data(data);            // print the data
  }
}
```



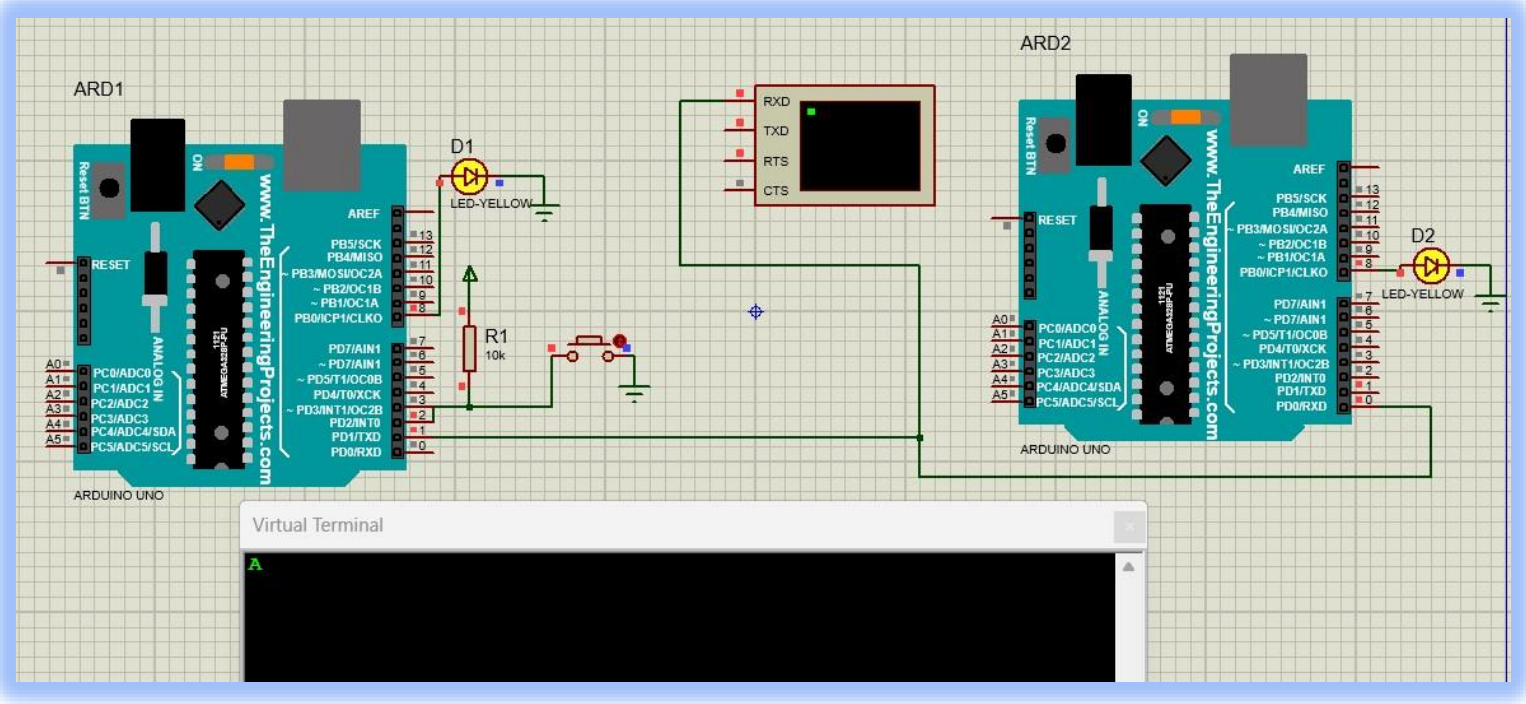
# -: USART Protocol :-

## /* 01. UART Transmitter Code (Sending 'A') */

```
void init_USART0(){
  volatile uint16_t UBRR_value = 103;  // UBBR0 = ((clock frequency/(16*baud rate)) - 1)
                                       // UBBR0 = ((16,000,000/(16*9600)) - 1)  =  103

  UBRR0H = (unsigned char)(UBRR_value >> 8);
  UBRR0L = (unsigned char) UBRR_value;          // Select the 9600 baud rate
  UCSR0B = (1 << RXEN0) | (1 << TXEN0);    // enable Tx and Rx
  UCSR0C = (1 << USBS0) | (3 << UCSZ00);   // set 2 stop bit & 8 data bit
}

void setup() {
  DDRD &= ~4;    // set port-D 2nd pin as Input
  DDRB = 1;      // set port-B 0th pin as Output

 init_USART0();

  while(1){
    if((PIND & 4)==0){
      PORTB ^= 1;

      while(!(UCSR0A & (1 << UDRE0))); // wait untill the UDRE0 bit is set
      UDR0 = 65;   // 65 = A
      for(volatile long i=0; i<100000; i++);  // switch debouncing delay
    }
  }
}

void loop() {}
```

## /* UART Receiver Code */

```
void init_USART0(){
  volatile uint16_t UBRR_value = 103;  // UBBR0 = ((clock frequency/(16*baud rate)) - 1)
                                       // UBBR0 = ((16,000,000/(16*9600)) - 1)  =  103

  UBRR0H = (unsigned char)(UBRR_value >> 8);
  UBRR0L = (unsigned char) UBRR_value;          // Select the 9600 baud rate
  UCSR0B = (1 << RXEN0) | (1 << TXEN0);    // enable Tx and Rx
  UCSR0C = (1 << USBS0) | (3 << UCSZ00);   // set 2 stop bit & 8 data bit
}

void setup() {
  DDRB = 1;  // port-B 0th bit set as Output

 init_USART0();
 volatile uint8_t receive_data;

  while(1){

     while(!(UCSR0A & (1 << RXC0))); // wait until the RXC0 bit is set
     receive_data = UDR0;

     if(receive_data == 'A'){
     PORTB ^= 1;
     }
  }
}

void loop() {}
```



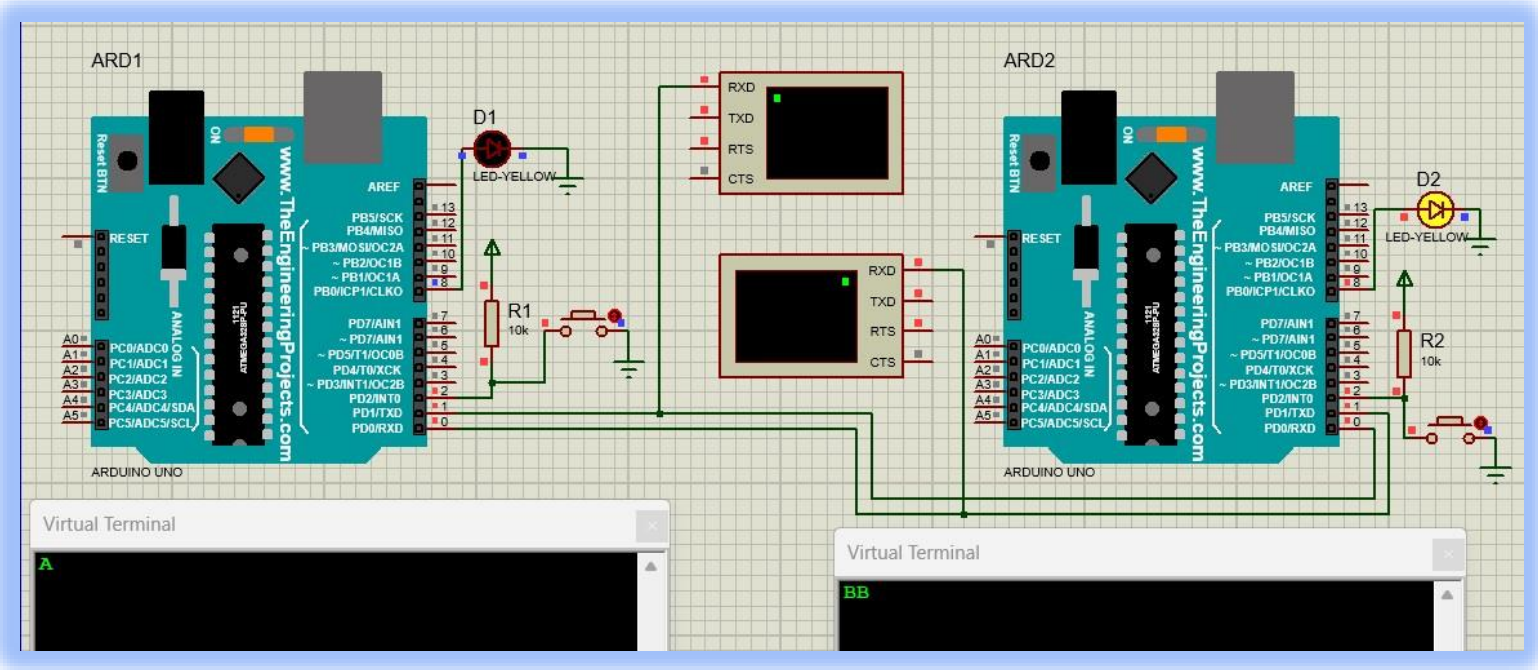
**/* 02. UART Transmit and Receive using Rx complete interrupt (Transmitter Code) */**

```
volatile uint8_t receive_data;

void init_USART0(){
  volatile uint16_t UBRR_value = 103;  // UBBR0 = ((clock frequency/(16*baud rate)) - 1)
                                       // UBBR0 = ((16,000,000/(16*9600)) -1)  =  103

  UBRR0H = (unsigned char)(UBRR_value >> 8);
  UBRR0L = (unsigned char) UBRR_value;          // Select the 9600 baud rate
  UCSR0B = (1 << RXEN0) | (1 << TXEN0) | (1<<RXCIE0); // enable Tx, Rx and Rx complete interrupt
  UCSR0C = (1 << USBS0) | (3 << UCSZ00);   // set 2 stop bit & 8 data bit
}

void USART_send(volatile char data){
  while(!(UCSR0A & (1 << UDRE0))); // wait untill the UDRE0 bit is set
  UDR0 = data;
}

void setup() {
  DDRD &= ~4;    // set port-D 2nd pin as Input
  DDRB = 1;      // set port-B 0th pin as Output

 init_USART0();

  while(1){
    if((PIND & 4)==0){
      USART_send('A');
      for(volatile long i=0; i<100000; i++);  // switch debouncing delay
    }

    if(receive_data == 'B'){
      PORTB ^= 1;
      receive_data=0;
    }
  }
}

ISR(USART_RX_vect){
  receive_data = UDR0;
}
```

**/* Receiver Code */**

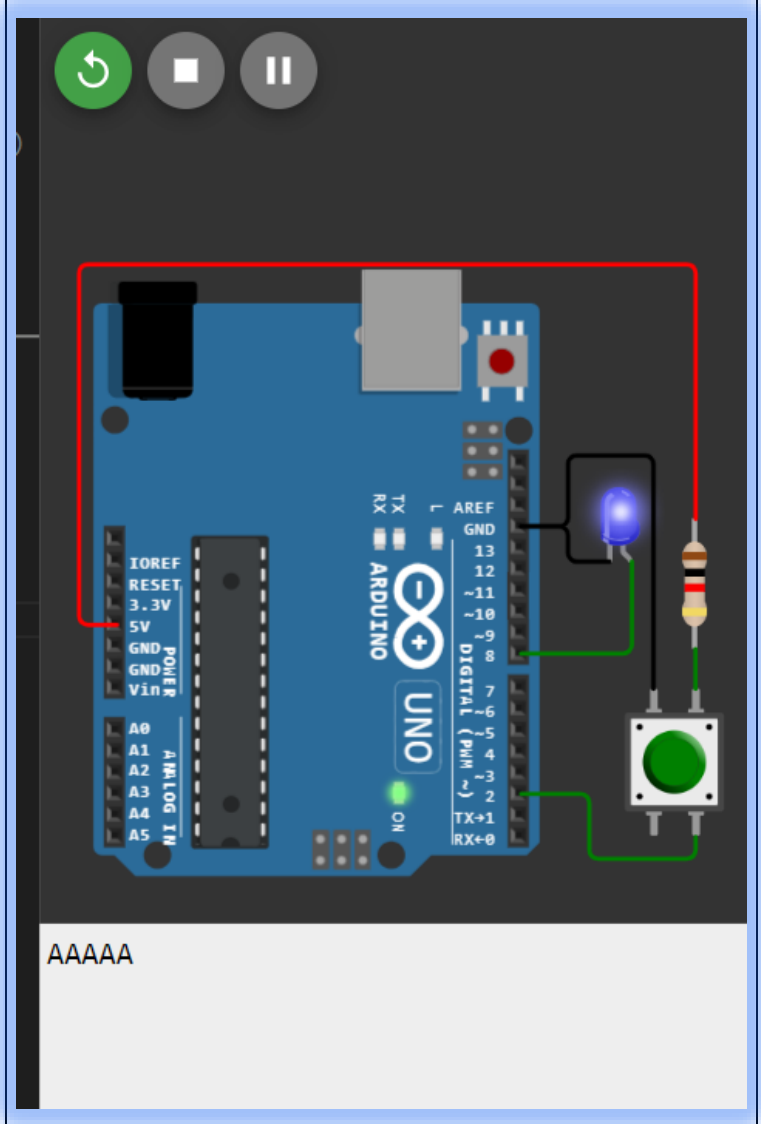
```
volatile uint8_t receive_data;

void init_USART0(){
  volatile uint16_t UBRR_value = 103;  // UBBR0 = ((clock frequency/(16*baud rate)) - 1)
                                       // UBBR0 = ((16,000,000/(16*9600)) -1)  =  103

  UBRR0H = (unsigned char)(UBRR_value >> 8);
  UBRR0L = (unsigned char) UBRR_value;          // Select the 9600 baud rate
  UCSR0B = (1 << RXEN0) | (1 << TXEN0) | (1<<RXCIE0); // enable Tx, Rx and Rx complete interrupt
  UCSR0C = (1 << USBS0) | (3 << UCSZ00);   // set 2 stop bit & 8 data bit
}

void USART_send(volatile char data){
  while(!(UCSR0A & (1 << UDRE0))); // wait untill the UDRE0 bit is set
  UDR0 = data;
}

void setup() {
  DDRD &= ~4;    // set port-D 2nd pin as Input
  DDRB = 1;  // port-B 0th bit set as Output

 init_USART0();

 while(1){
  if((PIND & 4)==0){
    USART_send('B');
    for(volatile long i=0; i<100000; i++);  // switch debouncing delay
  }

  if(receive_data == 'A'){
    PORTB ^= 1;
    receive_data=0;
  }
 }
}

ISR(USART_RX_vect){
  receive_data = UDR0;
}
```

This figure is the circuit diagram of the above code




## /* UART Transmitter Code */

```
void init_USART0(){
  volatile uint16_t UBRR_value = 103;  // UBBR0 = ((clock frequency/(16*baud rate)) - 1)
                                       // UBBR0 = ((16,000,000/(16*9600)) - 1)  =  103

  UBRR0H = (unsigned char)(UBRR_value >> 8);
  UBRR0L = (unsigned char) UBRR_value;          // Select the 9600 baud rate
  UCSR0B = (1 << RXEN0) | (1 << TXEN0);    // enable Tx and Rx
  UCSR0C = (1 << USBS0) | (3 << UCSZ00);   // set 2 stop bit & 8 data bit
}

void setup() {
  DDRD &= ~4;    // set port-D 2nd pin as Input
  DDRB = 1;      // set port-B 0th pin as Output

 init_USART0();

  while(1){
    if((PIND & 4)==0){
      PORTB ^= 1;

      while(!(UCSR0A & (1 << UDRE0))); // wait untill the UDRE0 bit is set
      UDR0 = 'A';
      for(volatile long i=0; i<100000; i++);  // switch debouncing delay
    }
  }
}

void loop() {}
```



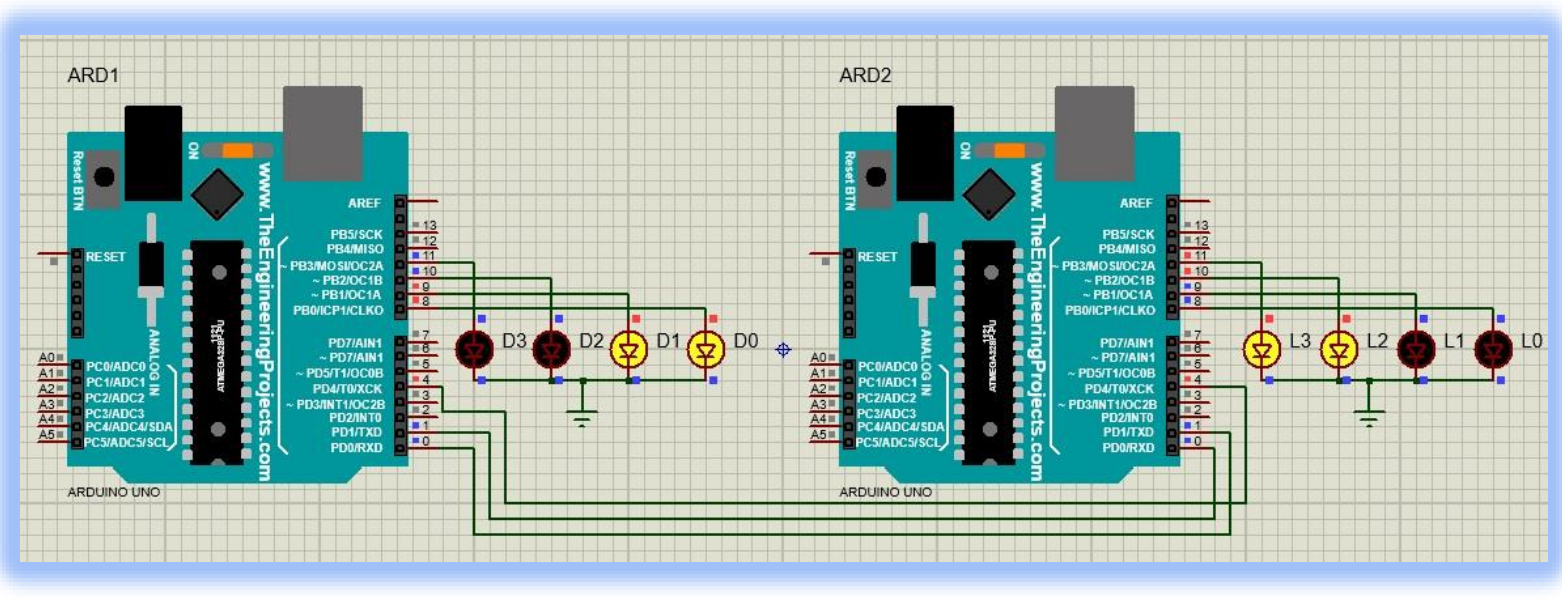
# -: USART Protocol in SPI Mode :-

## /* Master Code (only Master generates the clock (in XCK pin)) [Simple LED Output] using USART Rx interrupt */

```
volatile char receive_data;

void USART_Init(){
UBRR0 = 0;
DDRD |= 16;  //  XCK pin (i.e - PD4) set as output
UCSR0C =
(1<<UMSEL01)|(1<<UMSEL00)|(1<<UCPHA0)|(1<<UCPOL0);
UCSR0B = (1<<RXEN0)|(1<<TXEN0)|(1<<RXCIE0); // Enable
Tx, Rx & Recive complete interrupt
UBRR0 = 832;  // set baud rate as 9600
}


void USART_Transmit(volatile char data){
while (!(UCSR0A & (1<<UDRE0)));  // Wait for empty
transmit buffer
UDR0 = data;
}

void setup(){
  DDRB = 0x0f;
  USART_Init();

  while(1){
    USART_Transmit(0x0C);
    if(receive_data){
      PORTB = receive_data;
      receive_data = 0;
    }
  }
}

ISR(USART_RX_vect){
  receive_data = UDR0;
}
```

## /* Slave Code [Simple LED Output]*/

```
volatile char receive_data;

void USART_Init(){
UBRR0 = 0;
DDRD &= ~16;  //  XCK pin (i.e - PD4) set as Input as slave
UCSR0C =
(1<<UMSEL01)|(1<<UMSEL00)|(1<<UCPHA0)|(1<<UCPOL0);
UCSR0B = (1<<RXEN0)|(1<<TXEN0)|(1<<RXCIE0);  // Enable
Tx, Rx & Recive complete interrupt
UBRR0 = 832;  // set baud rate as 9600
}


void USART_Transmit(volatile char data){
while (!(UCSR0A & (1<<UDRE0)));  // Wait for empty
transmit buffer
UDR0 = data;
}

void setup(){
  DDRB = 0x0f;
  USART_Init();

  while(1){
    USART_Transmit(0x03);
    if(receive_data){
      PORTB = receive_data;
      receive_data = 0;
    }
  }
}

ISR(USART_RX_vect){
  receive_data = UDR0;
}
```

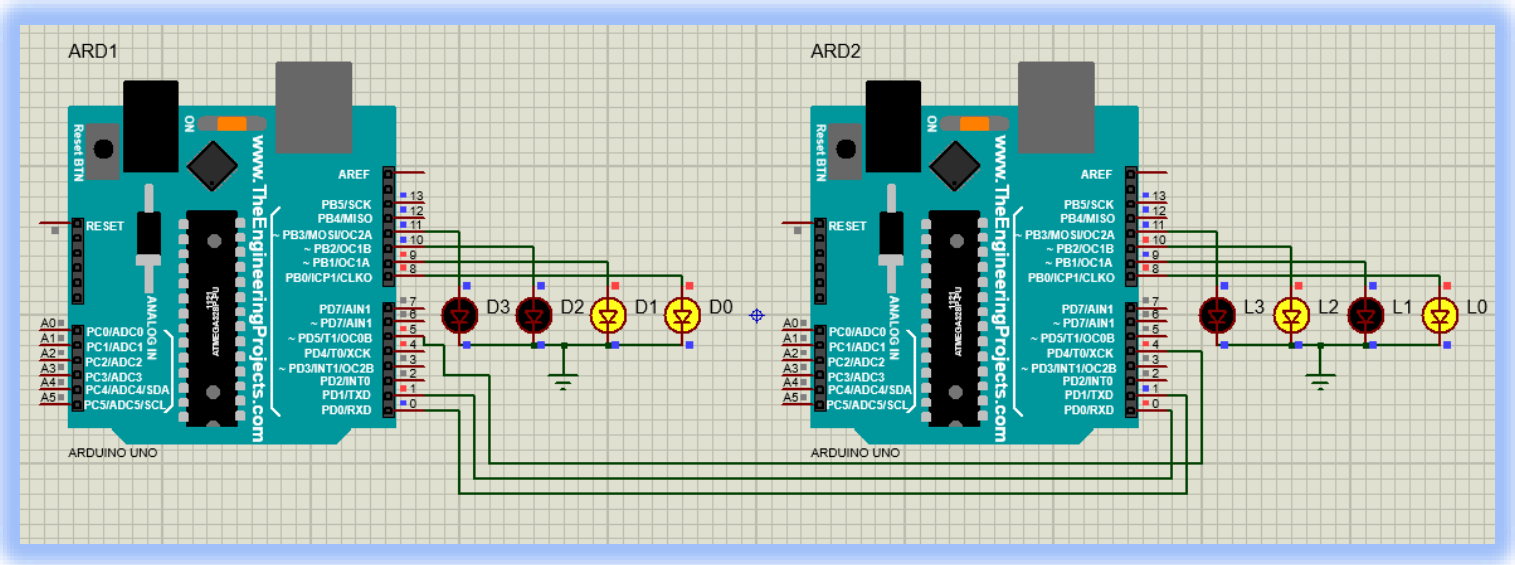


## /* This is the same code as above but without interrupt Master Code */

```
void USART_Init()
{
UBRR0 = 0;
DDRD |= 16;
UCSR0C =
(1<<UMSEL01)|(1<<UMSEL00)|(1<<UCPHA0)|(1<<UCPOL0);
UCSR0B = (1<<RXEN0)|(1<<TXEN0);
UBRR0 = 832;
}


volatile char USART_Tx_Rx(volatile char data){
while (!(UCSR0A & (1<<UDRE0))); /* Wait for empty transmit
buffer */
UDR0 = data;

while (!(UCSR0A & (1<<RXC0))); /* Wait for data to be
received */
return UDR0;
}

void setup(){
  DDRB = 0xff;
  USART_Init();

  while(1){
    PORTB = USART_Tx_Rx(0x05);
  }
}
```

## /* Slave Code */

```
void USART_Init(){
UBRR0 = 0;
DDRD |= 16;
UCSR0C =
(1<<UMSEL01)|(1<<UMSEL00)|(1<<UCPHA0)|(1<<UCPOL0);
UCSR0B = (1<<RXEN0)|(1<<TXEN0);
UBRR0 = 832;
}


volatile char USART_Tx_Rx(volatile char data){
while (!(UCSR0A & (1<<UDRE0))); /* Wait for empty transmit
buffer */
UDR0 = data;

while (!(UCSR0A & (1<<RXC0))); /* Wait for data to be
received */
return UDR0;
}

void setup(){
  DDRB = 0xff;
  USART_Init();

  while(1){
    PORTB = USART_Tx_Rx(0x03);
  }
}
```




---------- o ----------